

Buscador de Máximas Ocurrencias

Trabajo Practico Final

ALGORITMOS Y ESTRUCTURAS DE DATOS

Grupo: Alan Leslie · Axel Sanchez · Tomas Guerrero

Profesor: Leandro Caballero

Comision: 5

01 Introduccion

El objetivo del trabajo fue analizar archivos CSV y determinar cuales son los elementos que poseen la mayor cantidad de ocurrencias dentro del dataset. Al iniciar la aplicacion, el usuario selecciona un archivo CSV. Luego el sistema carga los datos en una lista de cadenas de texto (**List<string>**) y permite realizar busquedas utilizando dos estrategias:



Max-Heap

Busqueda utilizando una estructura Heap (Max-Heap).



Ordenamiento

Busqueda utilizando un metodo de ordenamiento alternativo.



Consultas

Comparacion de tiempos y visualizacion de la estructura Heap.

02 Metodos Implementados

BuscarConHeap()

- Cuenta las ocurrencias de cada elemento usando un diccionario.
- Construye una Max-Heap insertando cada elemento con su frecuencia.
- Extrae los N elementos con mayor cantidad de ocurrencias y los devuelve en *collected*.
- Utiliza FiltrarArriba (insercion) y FiltrarAbajo (extraccion) como auxiliares privados.
- Permite obtener los elementos mas frecuentes sin ordenar completamente los datos.

BuscarConOtro()

- Implementado con **Selection Sort** descendente sobre una lista de Dato.
- Cuenta las ocurrencias con un Dictionary, igual que BuscarConHeap.
- Ordena la lista de mayor a menor por frecuencia.
- Toma los primeros N elementos para devolverlos en *collected*.
- Solucion sencilla y facil de comprender. Complejidad $O(n^2)$.

Consulta1()

- Mide y compara los tiempos de BuscarConHeap y BuscarConOtro.
- Busca los 5 elementos con mayor cantidad de ocurrencias.
- Usa la clase **Stopwatch** de C# para medir milisegundos.
- Retorna ambos resultados en un unico string.

Consulta2()

- Construye la misma Max-Heap utilizada en el metodo de busqueda.
- Comienza en el nodo raiz y recorre sucesivamente los hijos izquierdos.
- Devuelve el recorrido desde la raiz hasta la hoja mas izquierda en formato texto.
- Usa flechas (↓) como separador visual entre nodos.

Consulta3()

- Construye la Max-Heap a partir de los datos de entrada.
- Muestra todos los elementos indicando el nivel de cada nodo.
- Aplica logica matematica: cuando contador == capActual, incrementa el nivel.
- Permite visualizar la estructura completa del arbol.

03 Problemas Encontrados y Soluciones

■ Consulta 2 — Multiples caminos de igual profundidad

Existian varios caminos con la misma profundidad. Se resolvio conservando el primer camino encontrado con profundidad maxima, simplificando la implementacion sin perder funcionalidad. Ademas, los nodos se mostraban uno al lado del otro; se incorpore una flecha (↓) como separador para representar visualmente el recorrido desde la raiz hasta la hoja.

■ Consulta 3 — Visualizacion de niveles

El desafio fue como indicar a que nivel pertenece cada nodo. Se resolvio con una logica matematica: al detectar que se completa un nivel (cuando contador == capActual), el sistema incrementa el contador de nivel y duplica la capacidad esperada para el proximo nivel.

■ Error en la lectura del CSV

El programa provisto por la catedra presenta un error: el CSV usa punto y coma (;) como separador, pero el Split esta definido por coma (,), lo que genera problemas en la mayoria de ejecuciones.

04 Posibles Mejoras

■ Optimizar algoritmos de busqueda Reemplazar Selection Sort por algoritmos de mayor eficiencia como MergeSort o QuickSort.	■ Visualizacion grafica del Heap Agregar una interfaz que muestre el arbol graficamente con nodos y relaciones.
■ Soporte para multiples CSV Permitir cargar y comparar estadisticas entre distintos conjuntos de datos.	■ Nuevas consultas estadisticas Agregar promedio de ocurrencias, histogramas y exportacion de informes.
■ Ampliar Consulta 2 Mostrar todos los caminos de igual profundidad, no solo el primero encontrado.	■ Corregir lectura del CSV Adaptar el Split segun el separador real del archivo (coma o punto y coma).

05 Conclusion

La realizacion de este trabajo permitio aplicar de manera practica los conceptos estudiados en la materia, especialmente el uso de estructuras de datos como Heap, algoritmos de ordenamiento y diccionarios. Tambien permitio comprender las diferencias de rendimiento entre distintos algoritmos y reforzar conocimientos relacionados con complejidad, ordenamiento y busqueda de informacion.

La experiencia fue util para comprender como utilizar estructuras de datos en problemas reales y desarrollar aplicaciones completas en C#. El contraste entre la eficiencia del Max-Heap ($O(n \log n)$) y el Selection Sort ($O(n^2)$) resulto especialmente ilustrativo en terminos practicos.

Max-Heap	$O(n \log n)$ — eficiente para grandes datasets
Selection Sort	$O(n^2)$ — simple pero mas lento
Dictionary	$O(1)$ promedio — conteo de ocurrencias optimo
Stopwatch	Medicion objetiva de tiempos de ejecucion